

Python

Summary Session 16



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Что такое регулярные выражения и для чего они используются?



Регулярные выражения (RegEx)

Это инструмент для поиска, сравнения, замены и проверки строк по заданному шаблону

Назначение регулярных выражений



Поиск текста по шаблону



Проверка формата строки



Замена текста



Разделение строк

Области применения регулярных выражений



Обработка данных



Формы ввода



Парсинг текста



Поиск в коде



Модуль re

Этот модуль предоставляет инструменты для работы с регулярными выражениями

Модуль re: подключение

```
import re
```

Экспресс-опрос

?

Назовите основные функции модуля ге

Основные функции модуля re

`re.search(pattern, string)`

Ищет первое совпадение шаблона
в строке

`re.match(pattern, string)`

Проверяет, начинается ли строка с
шаблона

`re.findall(pattern, string)`

Возвращает список всех
совпадений

`re.finditer(pattern, string)`

Возвращает итератор с
совпадениями

`re.sub(pattern, repl, string)`

Заменяет найденные совпадения

`re.split(pattern, string)`

Разделяет строку по шаблону



Функция findall

Эта функция ищет все совпадения шаблона в строке и возвращает их в виде списка

Функция findall



Синтаксис

```
re.findall(pattern, string)
```

Пояснения

- `pattern` – регулярное выражение (шаблон поиска)
- `string` – строка, в которой выполняется поиск

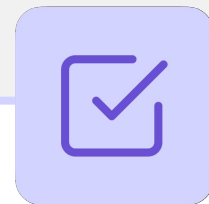
Пример: поиск всех чисел в строке

```
import re

text = "Python 3.9, Java 17, C++ 14"

numbers = re.findall(r"\d+", text) # Поиск всех чисел
print(numbers)
```

Важно



Регулярные выражения используют специальные символы для **поиска букв, цифр, пробелов и других элементов текста**

Экспресс-опрос

?

Какие специальные символы вы помните?

Обозначения основных символов

Символ	Описание
\d	Любая цифра (0-9)
\D	Любой символ, кроме \d
\w	Буквы, цифры и _
\W	Любой символ, кроме \w
\s	Пробел, \t, \n
\S	Любой символ, кроме \s
.	Любой символ, кроме \n

Пример использования специальных СИМВОЛОВ

```
text = "\tPython 3.12, Java 17, C++ 14!\n"

print("Цифры:", re.findall(r"\d", text))
print("Двузначные цифры:", re.findall(r"\d\d", text))

print("НЕ цифры:", re.findall(r"\D", text))

print("Буквы, цифры, _:", re.findall(r"\w", text))

print("НЕ буквы, цифры, _:", re.findall(r"\W", text))

print("Пробелы:", re.findall(r"\s", text))

print("НЕ пробелы:", re.findall(r"\S", text))

print("Все символы (кроме \n):", re.findall(r".", text))
```

Экспресс-опрос

?

Для чего рекомендуется ставить `г` перед строкой?

Важно



В регулярных выражениях **используется много специальных символов**, таких как `\d`, `\w`, `\s` и другие.

Python обрабатывает `\` (**обратный слэш**) как **управляющий символ**, из-за чего могут возникнуть ошибки. Чтобы избежать этого, перед строкой **рекомендуется ставить `r`** ("**сырая**" строка, **raw string**).

Пример: если не использовать r

```
pattern = "\d" # ОШИБКА: \d будет воспринято как управляющий символ
print(re.findall(pattern, "Price: 123"))
```

```
/home/tanya/PycharmProjects/pythonProgramItch/.venv/bin/python /home/tanya/PycharmProjects/pythonProgramItch
['1', '2', '3']
/home/tanya/PycharmProjects/pythonProgramItch/_notes/test.py:3: SyntaxWarning: invalid escape sequence '\d'
  pattern = "\d" # ОШИБКА: \d будет воспринято как управляющий символ

Process finished with exit code 0
```

Экспресс-опрос



Что такое классы символов?



Классы символов

Они обозначаются квадратными скобками `[]` и используются для поиска любого из указанных внутри символов. Они позволяют задать набор символов, который должен встречаться в искомом фрагменте

Внутри [] можно использовать

Символ	Пояснение	Назначение	Пример
-	дефис	для указания диапазона символов	[a-z] — все буквы от a до z
^	каретка	для исключения символов	[^0-9] — всё, кроме цифр

Обозначения классов символов

Запись	Описание
[afc]	Один из символов a, f или c
[0-9]	Любая цифра от 0 до 9 (аналог \d)
[a-z]	Любая строчная буква от a до z
[A-Z]	Любая заглавная буква от A до Z
[a-zA-Z]	Любая буква (заглавная или строчная)
[^abc]	Любой символ, кроме a, b или c
[^0-9]	Любой символ, кроме цифры

Пример использования класса СИМВОЛОВ

```
text = "Report, report, report2, report10"

print("Буквы r или R в слове:", re.findall(r"[rR]eport", text))

print("Все цифры:", re.findall(r"[0-9]", text))

print("Заглавные буквы:", re.findall(r"[A-Z]", text))

print("Строчные буквы:", re.findall(r"[a-z]", text))

print("Все буквы:", re.findall(r"[a-zA-Z]", text))

print("Все, кроме цифр:", re.findall(r"^[0-9]", text))
```



Квантификаторы

Квантификаторы в регулярных выражениях определяют количество повторений символов или групп. Они позволяют указывать, сколько раз подряд должен встречаться символ или шаблон

Экспресс-опрос

?

Какие квантификаторы вы помните?

Обозначения квантификаторов

Квантификатор	Описание
+	Один или более раз (1, 2, 3...)
*	Ноль или более раз (0, 1, 2...)
?	Ноль или один раз (0, 1)
{n}	Ровно n раз
{n, }	n и более раз
{n, m}	От n до m раз

Пример использования квантификаторов

```
text = """
Orders: ID123, ID4567, ID89
Numbers: 123-45-67, 321-45-67
Prices: 100$, 199.50$, 99.99€, 0.49€, .99€
File names: report.txt, report2.txt, report10.txt
"""

print("Одна или более цифр:", re.findall(r"\d+", text))

print("Телефонные номера (формата xxx-xx-xx):", re.findall(r"\d{3}-\d{2}-\d{2}", text))

print("Цены (числа с десятичной точкой):", re.findall(r"\d+\.\d+", text))

print("ID-коды:", re.findall(r"ID\d{2,}", text))

print("Имена файлов 0+ цифр:", re.findall(r"report\d*.txt", text))

print("Имена файлов 0/1 цифр:", re.findall(r"report\d?.txt", text))

print("Имена файлов 1/2 цифр:", re.findall(r"report\d{1,2}.txt", text))
```

Экспресс-опрос

?

Что такое жадные и ленивые квантификаторы?

Важно



Квантификаторы (*, +, {n,m}) по умолчанию работают жадно – они стараются захватить как можно больше символов.

Но иногда нужно, чтобы они захватывали минимально возможное количество символов – в этом случае используются ленивые квантификаторы.

Важно



Чтобы сделать квантификатор **ленивым**, нужно добавить ? после него

Обозначения жадных и ленивых квантификаторов

Квантификатор	Тип	Описание
*	Жадный	Захватывает максимально возможное количество символов
*?	Ленивый	Захватывает минимально возможное количество символов
+	Жадный	Захватывает минимум 1 символ , но как можно больше
+	Ленивый	Захватывает минимум 1 символ , но как можно меньше
{n, m}	Жадный	Захватывает от n до m , но как можно больше
{n, m}?	Ленивый	Захватывает от n до m , но как можно меньше

Пример

```
text = "<div>Hello</div><div>World</div>"

greedy = re.findall(r"<.*>", text)  # Жадный
lazy = re.findall(r"<.*?>", text)   # Ленивый

print(greedy)
print(lazy)
```

Экспресс-опрос

?

Как можно экранировать специальные символы, чтобы использовать их как обычные?

Специальные символы



В регулярных выражениях есть символы, которые имеют особое значение (. + * {} [] () | ^ \$).

Если нужно найти их как обычные символы, их нужно экранировать, добавляя \ перед символом.

Пример экранирования

```
text = "report.txt, report2.txt, report10.txt, some_txt_report,
some_report_txt, reports\\report.txt, report2.txt"

print("Имена файлов с txt:", re.findall(r"\w+.txt", text))

# Имена файлов с расширением .txt
print("Имена файлов с расширением .txt:", re.findall(r"\w+\.txt", text))

# Имена файлов в папке
print("Имена файлов в папке:", re.findall(r"\w+\\w+\.w+", text))
```

Экспресс-опрос



Что такое якоря?



Якоря

Якоря используются в регулярных выражениях для указания позиции совпадения в строке.

Они не ищут символы, а определяют, где именно должен находиться искомый фрагмент.

Обозначения якорей

Якорь	Описание
^	Начало строки
\$	Конец строки
\b	Граница слова
\B	Не граница слова

Пример использования якорей

```

text = "Hello world! Welcome to world"

print("Слово в начале строки:", re.findall(r"^\\w+", text))
print("Слово в конце строки:", re.findall(r"\\w+$", text))

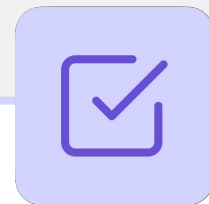
text2 = "category wildcat education _cat_ catalog"

print("Слова с 'cat' внутри:", re.findall(r"\\w+cat\\w+", text2))
print("Слова с 'cat' в начале слов:", re.findall(r"\\bcat\\w*", text2))
print("Слова с 'cat' в конце слов:", re.findall(r"\\w*cat\\b", text2))

text3 = "X123X 234 4567X X999"
print("Числа внутри строк:", re.findall(r"\\B\\d+\\B", text3))

```

Альтернативы



Оператор | (ИЛИ) позволяет искать один из нескольких вариантов.

Пример

```
text = "Meeting on 2024-05-10 or 10/05/2024 at 14:30"

# Найдём даты в формате YYYY-MM-DD или DD/MM/YYYY
print("Даты:", re.findall(r"(\d{4}-\d{2}-\d{2}|\d{2}/\d{2}/\d{4})",
text))
```

Экспресс-опрос

?

Как обозначаются группы?



Группы

Группы обозначаются круглыми скобками () и используются для объединения нескольких символов в одну логическую часть.

Они позволяют извлекать части совпадений, применять квантификаторы к целым выражениям и работать с подстроками отдельно

Извлечение данных с помощью групп



Пример

```
text = "Order ID: 12345, Invoice No: 67890"
```

```
# Найдём ID заказа и счёта
```

```
match = re.search(r"Order ID: (\d+),  
Invoice No: (\d+)", text)
```

```
if match:  
    print("ID заказа:", match.group(1))  
    print("Номер счёта:", match.group(2))
```

Пояснения

Функции `re.search()` и `re.match()` позволяют **доступ к частям совпадений** через `group()`

Использование негруппирующих скобок



Пример

```
text = "USD 100, EUR 200, GBP 300"

# Найдём суммы, не выделяя валюту
matches = re.findall(r"(?:USD|EUR|GBP) (\d+)",
text)
print("Суммы:", matches)

# Найдём суммы и валюту
matches = re.findall(r"(USD|EUR|GBP) (\d+)",
text)
print("Суммы:", matches)
```

Пояснения

Если группа нужна для логики, но не для извлечения данных, можно использовать `(?:...)`

Экспресс-опрос

?

Что делает функция `re.match()`?



Функция `re.match()`

Эта функция проверяет, начинается ли строка с заданного шаблона.

Если совпадение найдено, функция возвращает объект `Match`, который содержит информацию о совпадении, иначе `None`

Важно



Объект Match содержит **информацию о найденном фрагменте**, включая:

- `.group()` – само совпадение.
- `.start()` – индекс начала совпадения.
- `.end()` – индекс конца совпадения.
- `.span()` – кортеж `(start, end)`, показывающий границы совпадения

Пример

```
text = "ID12345 is confirmed. ID23456 is confirmed"

# Проверяем, начинается ли строка с "ID" + цифры
match = re.match(r"ID\d+", text)

if match:
    print("Объект Match:", match)
    print("Само совпадение:", match.group())
    print("Диапазон совпадения:", match.span())
else:
    print("Нет совпадения.")
```

Экспресс-опрос

?

Какой объект возвращает функция `re.search()`?



Функция `re.search()`

Эта функция ищет первое совпадение регулярного выражения в любой части строки и возвращает объект `Match`

Пример

```
text = "Order ID: 12345, Invoice No: 67890, Ref: ABC9876"
```

```
# Найдём первое число в тексте
```

```
match = re.search(r"\d+", text)
```

```
if match:
```

```
    print("Объект Match:", match)
```

```
    print("Само совпадение:", match.group())
```

```
    print("Индекс начала:", match.start())
```

```
    print("Индекс конца:", match.end())
```

```
    print("Диапазон совпадения:", match.span())
```

```
else:
```

```
    print("Нет совпадения.")
```


Экспресс-опрос

?

Какой флаг делает поиск регистронезависимым?



Флаг re.IGNORECASE

Этот флаг сокращённо обозначается `re.I`, он делает поиск регистронезависимым — шаблон будет находить совпадения в любом регистре, даже если написан в нижнем или верхнем

Пример

```

text = "Python is popular."

# Найдём слово "python" без учёта регистра
match = re.search(r"python", text, re.IGNORECASE)

if match:
    print("Найдено:", match.group())
    
```

Экспресс-опрос

?

Что делает функция `re.finditer()`?



Функция `re.finditer()`

Эта функция ищет все совпадения регулярного выражения в строке и возвращает итератор объектов `Match`

Пример

```
text = "Order ID: 12345, Invoice No: 67890, Ref: ABC9876"
```

```
# Найдём все числа в тексте
```

```
matches = re.finditer(r"\d+", text)
```

```
for match in matches:
```

```
    print("Объект Match:", match)
```

```
    print("Само совпадение:", match.group())
```

```
    print("Диапазон совпадения:", match.span())
```

```
    print()
```



Функция `re.split()`

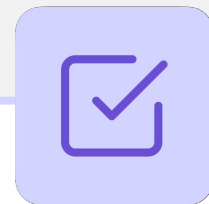
Эта функция разделяет строку на части, используя регулярное выражение как разделитель

Экспресс-опрос

?

В чем разница между функциями `re.split()` и `str.split()`?

Важно



Функция `re.split()` работает аналогично `str.split()`, но позволяет **разбивать текст по сложным шаблонам**, а не только по одному символу

Пример

```
text = """Python is popular. It is used in web development, data science,  
and automation. Many developers choose Python for its simplicity."""
```

```
# Разделяем строку по запятым, пробелам и точкам
```

```
words = re.split(r"[,\s.]+", text)
```

```
print("Список слов:", words)
```

Экспресс-опрос

?

Какая функция заменяет все найденные совпадения на указанную строку или результат функции?



Функция `re.sub()`

Эта функция заменяет все найденные совпадения на указанную строку или результат функции. Это полезно для форматирования текста, удаления лишних символов и исправления данных

Функция re.sub()



Синтаксис

```
re.sub(pattern, repl, string)
```

Пояснения

- pattern – шаблон для поиска
- repl – строка, на которую будет заменено совпадение
- string – исходный текст

Пример

```
text = "apple,  banana ,  orange ,grape"  
  
# Удаляем лишние пробелы перед и после запятыми  
clean_text = re.sub(r"\s*,\s*", ", ", text)  
  
print("Отформатированный текст:", clean_text)
```



Объектно-ориентированное программирование (ООП)

Это подход к написанию программ, в котором основное внимание уделяется объектам и их взаимодействию

ООП



ООП описывает сущности реального мира в виде объектов, которые объединяют поля (данные, состояние) и методы (поведение). Это способ думать о программе как о взаимодействии объектов, у каждого из которых есть своя роль и ответственность.

Экспресс-опрос



Что такое класс?



Класс

Это шаблон (чертёж), по которому создаются объекты

Класс



Класс описывает

- Какие **данные** будут у объектов (например, имя, возраст, цена)
- Какие **методы** можно будет с ними выполнять

Пример

Класс Book может описывать, что у книги есть `title`, `author`, и метод `show_description()`



Объект

Это конкретный экземпляр класса, созданный по его шаблону.

Объект



Описание

У него уже есть конкретное состояние и возможность выполнять определённые действия

Пример

`book1 = Book("1984", "Оруэлл")` — объект создаётся из класса `Book`, и у него своё конкретное название и автор

Парадигмы программирования



Почти все, что мы изучали ранее, относится к **процедурному программированию**, при котором программа представляет собой последовательность инструкций, иногда объединённых в процедуры и функции. Две другие самые популярные парадигмы — это **объектно-ориентированное программирование (ООП)** и **функциональный подход**.

Экспресс-опрос

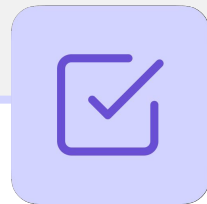
?

Для чего лучше подходит ООП, а для чего – функциональное программирование?

ООП vs функциональный подход

Объектно-ориентированный подход	Функциональный подход
Основан на объектах и классах	Основан на функциях как основных строительных блоках
Основные единицы — это объекты , которые хранят состояние и имеют поведение	Данные и поведение строго разделены: функции обрабатывают данные, не изменяя их
Программа строится как система взаимодействующих объектов	Часто используются чистые функции — одинаковый результат при одинаковом вводе
Удобен, когда нужно моделировать сущности реального мира	Предпочтение отдаётся неизменяемым структурам и отсутствию побочных эффектов
Хорошо подходит для сложных, расширяемых программ	Хорошо подходит для математических расчётов, анализа данных, обработки потоков информации

Важно



В Python можно использовать оба подхода, и часто они дополняют друг друга. ООП хорошо подходит для организации логики и структуры программы, а функциональные элементы — для коротких и чистых операций с данными.

Преимущества ООП



Позволяет структурировать сложные программы



Упрощает моделирование реальных сущностей



Повышает переиспользуемость и читаемость кода



Облегчает расширение и сопровождение программ

Создание класса



Синтаксис

```
# Пустой класс без полей и методов  
class ClassName:  
    pass
```

Пояснение

Для создания класса используется ключевое слово `class`, за которым следует имя класса

Экспресс-опрос

?

Какие правила именования классов вы помните?

Правила именования классов



Имя класса пишется с заглавной буквы



Если имя состоит из нескольких слов, используется стиль CamelCase



Не стоит использовать имена, совпадающие с встроенными типами: list, str, object и т.д.



Магический метод `__init__()`

Это инициализатор, который автоматически вызывается при создании объекта.
Он отвечает за инициализацию объекта — то есть за установку его начального состояния

Важно



- **Конструктор** — это метод `__new__()`, который создаёт, т.е. конструирует объект (также вызывается автоматически)
- **Инициализатор** — это метод `__init__()` — который инициализирует поля в уже созданном объекте

__init__()



Синтаксис

```
class ClassName:
    def __init__(self, arg1, arg2, ...):
        self.attr1 = arg1
        self.attr2 = arg2
        ...
```

Пояснение

- `arg1, arg2` — значения, передаваемые при создании объекта
- `attr1, attr2` — поля или свойства объекта, сохраняемые через `self`
- `self` — только что созданный объект, которому будут присвоены поля

__init__()



Пример

```
class Book:
    def __init__(self, title, author): #
        # ожидаемые данные для книги
        self.title = title # сохраняем
        # название книги в объекте
        self.author = author # сохраняем
        # имя автора
```

Пояснение

- Метод `__init__()` вызывается автоматически при создании объекта
- `self` — ссылка на сам объект, которому будет присвоено состояние
- `self.title` и `self.author` — это **поля или свойства объекта**

Экспресс-опрос

?

Назовите особенности метода `__init__`?

Особенности метода `__init__`

- 
 Называется строго `__init__`
- 
 Вызывается автоматически при создании объекта
- 
 Обязан принимать первым аргументом `self` — ссылку на текущий объект
- 
 Через `self` задаются поля объекта — переменные, которые будут ему "принадлежать"
- 
 Может отсутствовать, если класс не имеет состояния

Экспресс-опрос

?

Что такое поля, методы и атрибуты?



Поля

Это переменные, которые хранят состояние объекта. Они задаются при создании объекта, обычно через метод `__init__`, и сохраняются внутри самого объекта.

Пример

```
class Book:
    def __init__(self, title, author):
        self.title = title # поле
        self.author = author # поле
```



Методы

Это функции, определённые внутри класса. Они описывают, что умеет делать объект: например, выводить информацию, менять значения полей, выполнять расчёты.

Пример

```
class Book:
    def __init__(self, title, author): # метод
        self.title = title # поле
        self.author = author # поле
```




Атрибут

Это общее название для любой части объекта, доступной через точку: `object.something`.

Экспресс-опрос

?

Как называется обязательный первый параметр в методах класса?



Параметр `self`

Это обязательный первый параметр в методах класса, который является ссылкой на объект.

Важно



self не является зарезервированным словом, но принято использовать именно его

Зачем нужен self



Позволяет сохранить данные внутри объекта



Даёт доступ к полям и методам объекта



Отличает свойства объекта от обычных переменных внутри метода

Пример: сохраняем данные в объекте



Код

```
class Book:
    def __init__(self, book_title,
book_author): # self - созданный объект
        self.title = book_title #
        # присвоение переменной title объекту
        self.author = book_author #
        # присвоение переменной author объекту
```

Пояснение

Без self переменные title и author были бы локальными переменными метода и исчезли бы после выхода из метода

Важно



Каждый объект класса — это независимый экземпляр, созданный по шаблону класса, но со своими значениями.

Создание объекта



Синтаксис

```
object_name = ClassName(arg1, arg2, ...)
```

Пояснение

- `ClassName` — имя класса, по шаблону которого создаётся объект
- `arg1, arg2` — значения, которые ожидаются в методе `__init__`
- `object_name` — переменная, в которую сохраняется созданный объект

Пример

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

# Создание книг
book1 = Book("1984", "George Orwell")
book2 = Book("Brave New World", "Aldous Huxley")
```

Экспресс-опрос

?

Что происходит при создании объекта?

Что происходит при создании объекта



Python вызывает метод `__new__()` — создаёт пустой объект



Затем вызывает `__init__()` — инициализирует поля объекта (через `self`)



Возвращается готовый объект, с которым можно работать

Доступ к полям объекта



Синтаксис

`object_name.attribute`

Пояснение

- `object_name` — имя переменной, в которой хранится ссылка на объект
- `attribute` — имя поля, к которому обращаемся

Пример: доступ к полям

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("1984", "George Orwell")

print(book.title)
print(book.author)
```

Пример: изменение значения поля

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("1984", "George Orwell")

print(book.title)  # старое имя
print(book.author)

book.title = "Animal Farm"  # изменение значения поля title
print(book.title)  # измененное имя
```

Важно



Каждое поле относится конкретно к своему объекту — изменение в одном объекте не влияет на другие



Методы

Это функции, определённые внутри класса. Они описывают поведение объекта: что он умеет делать, как работает с данными, которые в нём хранятся.

Методы класса



Синтаксис

```
class ClassName:
    def method_name(self, ...):
        ...
```

Пояснение

- Метод обязательно принимает первым параметром `self` — это ссылка на текущий объект
- Через `self` метод может обращаться к полям объекта и вызывать другие его методы

Пример

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def get_description(self): # self - ссылка на объект, у которого вызван метод
        return f"{self.title} - {self.author}"

    def show_info(self): # self - ссылка на объект, у которого вызван метод
        print(self.get_description()) # вызов другого метода через self

    def make_author_upper(self): # self - ссылка на объект, у которого вызван метод
        self.author = self.author.upper()

book = Book("1984", "George Orwell")
book.show_info() # вызов метода у объекта
book.make_author_upper() # изменение объекта с помощью метода
book.show_info() # вывод нового состояния
```

Экспресс-опрос

?

Назовите особенности методов

Особенности методов



Методы вызываются через точку: `object.method()`



При вызове `book.show_info()` Python автоматически подставляет ссылку из переменной `book` в переменную `self`



Методы могут не только выводить информацию, но и **изменять поля**, выполнять расчёты, возвращать значения и т.д.



ВОПРОСЫ





ЗАДАНИЕ





Выберите верный вариант ответа

1.1 Что делает следующий код?

```
import re
```

```
text = "Version 3.10, Build 2023-04-01, Update 12.5.1"
```

```
matches = re.findall(r"\d+\.\d+", text)
print(matches)
```

- a. Находит только целые числа
- b. Находит все группы вида число.число
- c. Находит все группы вида число.число.число
- d. Находит все группы вида число любой символ число



Выберите верный вариант ответа

1.1 Что делает следующий код?

```
import re
```

```
text = "Version 3.10, Build 2023-04-01, Update 12.5.1"
```

```
matches = re.findall(r"\d+\.\d+", text)
print(matches)
```

- a. Находит только целые числа
- b. Находит все группы вида число.число**
- c. Находит все группы вида число.число.число
- d. Находит все группы вида число любой символ число



Выберите верный вариант ответа

1.2 Что напечатает следующий код?

```
import re
text = "ID123, ID56, ID, ID78910"

print(re.findall(r"ID\d{2,4}", text))
```

- a. ['ID123', 'ID56']
- b. ['ID123', 'ID56', 'ID7891']
- c. ['ID123', 'ID56', 'ID', 'ID78']



Выберите верный вариант ответа

1.2 Что напечатает следующий код?

```
import re
text = "ID123, ID56, ID, ID78910"

print(re.findall(r"ID\d{2,4}", text))
```

- a. ['ID123', 'ID56']
- b. ['ID123', 'ID56', 'ID7891']
- c. ['ID123', 'ID56', 'ID', 'ID78']



Выберите верный вариант ответа

1.3 Какой из шаблонов найдёт ВСЁ, кроме букв?

- a. `\D+`
- b. `[^a-zA-Z]+`
- c. `\W+`
- d. `[^\s]+`



Выберите верный вариант ответа

1.3 Какой из шаблонов найдёт ВСЁ, кроме букв?

- a. `\D+`
- b. `[^a-zA-Z]+`
- c. `\W+`
- d. `[^\s]+`



Выберите верный вариант ответа

1.4 Что найдёт шаблон `\B\d+\B`?

- a. Все числа в начале строки
- b. Числа, не стоящие на границе слова
- c. Все числа
- d. Числа, стоящие в начале слова



Выберите верный вариант ответа

1.4 Что найдёт шаблон `\B\d+\B`?

- a. Все числа в начале строки
- b. Числа, не стоящие на границе слова
- c. Все числа
- d. Числа, стоящие в начале слова



Выберите верный вариант ответа

1.5 Что делает `re.compile()`?

- a. Запускает регулярное выражение
- b. Сохраняет шаблон для повторного использования



Выберите верный вариант ответа

1.5 Что делает `re.compile()`?

- a. Запускает регулярное выражение
- b. Сохраняет шаблон для повторного использования



ВОПРОСЫ





Выберите верный вариант ответа

2.1 Что будет выведено?

```
class Book:
    def __init__(self, book_name):
        self.title = book_name
```

```
book = Book("1984")
print(book.title)
```

- a. book_name
- b. 1984
- c. title
- d. Ошибка



Выберите верный вариант ответа

2.1 Что будет выведено?

```
class Book:
    def __init__(self, book_name):
        self.title = book_name
```

```
book = Book("1984")
print(book.title)
```

- a. book_name
- b. 1984**
- c. title
- d. Ошибка



Выберите верный вариант ответа

**2.2 Укажите, что подходит под определение
"поле объекта":**

- a. Значение, сохранённое в классе
- b. Значение, сохранённое в объекте через self
- c. Имя класса
- d. Аргумент конструктора, не сохранённый в объекте



Выберите верный вариант ответа

2.2 Укажите, что подходит под определение "поле объекта":

- a. Значение, сохранённое в классе
- b. Значение, сохранённое в объекте через self
- c. Имя класса**
- d. Аргумент конструктора, не сохранённый в объекте



Выберите верный вариант ответа

2.3 Что можно отнести к атрибутам объекта?

- a. Только поля
- b. Только методы
- c. Поля и методы



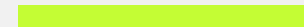
Выберите верный вариант ответа

2.3 Что можно отнести к атрибутам объекта?

- a. Только поля
- b. Только методы
- c. Поля и методы



Выполните задание



2.4 Найдите ошибку в коде:

```
class Person:
    def __init__(name, age):
        self.name = name
        self.age = age
```




Выполните задание

2.4 Найдите ошибку в коде:

```
class Person:
    def __init__(name, age):
        self.name = name
        self.age = age
```

Ответ: В методе `__init__` обязательно должен быть первым параметром `self`

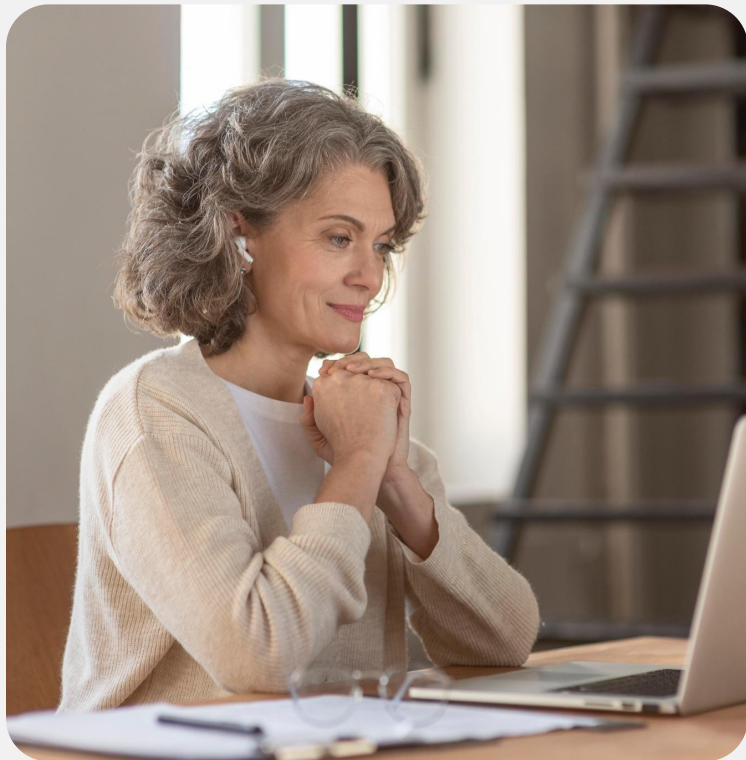


Выполните задание

2.5 Найдите ошибку в коде:

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book
print(book.title)
```



Выполните задание

2.5 Найдите ошибку в коде:

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book
print(book.title)
```

Ответ: Book — это ссылка на класс, а не создание объекта. Нужно: book = Book("...", "...")



Выберите верный вариант ответа

2.6 Выберите верное утверждение:

- a. Метод `__init__` вызывается вручную при создании объекта
- b. Все объекты класса обязательно имеют одинаковые значения полей
- c. Поля объекта доступны только внутри `__init__`
- d. Объекты одного класса могут хранить разные значения полей



Выберите верный вариант ответа

2.6 Выберите верное утверждение:

- a. Метод `__init__` вызывается вручную при создании объекта
- b. Все объекты класса обязательно имеют одинаковые значения полей
- c. Поля объекта доступны только внутри `__init__`
- d. Объекты одного класса могут хранить разные значения полей



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



1. Удаление HTML-тегов



Реализуйте программу, которая должна удалить все HTML-теги из строки.

Данные:

```
text = "<div>Hello, <strong>world</strong>!</div>"
```

Пример вывода:

```
Hello, world!
```


2. Класс Circle



Создайте класс `Circle`, описывающий окружность.

- У каждого объекта должно быть поле `radius`.
- Добавьте метод `get_area()` для вычисления площади.
- Создайте окружность и выведите её площадь.
- Измените радиус и снова выведите площадь.

Пример вывода:

Площадь: 78.5

Новая площадь: 201.0



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Извлечение дат

Реализуйте программу, которая должна:

- Найти в тексте все даты в форматах DD/MM/YYYY, DD-MM-YYYY и DD.MM.YYYY.

Данные:

```
text = "The events N 123456 happened on
15/03/2025, 01.12.2024 and 09-09-2023.
Deadline: 28/02/2022."
```

Пример вывода:

15/03/2025

01.12.2024

09-09-2023

28/02/2022

Домашнее задание

2. Разделение списка тегов

Реализуйте программу, которая должна:

- Прочитать строку с тегами, введёнными пользователем.
- Разделить её на отдельные теги, независимо от того, чем они были разделены (запятые, точки с запятой, слэши или пробелы).
- Удалить лишние пробелы и пустые значения.

Данные:

```
tag_input = "python, data-science / machine-learning; AI  neural-networks"
```

Пример вывода:

```
['python', 'data-science', 'machine-learning', 'AI', 'neural-networks']
```



ВОПРОСЫ



Домашнее задание

1. Класс Rectangle

Создайте класс `Rectangle`, который описывает прямоугольник.

- У каждого объекта должны быть два поля: `width` и `height`.
- Добавьте метод `get_area()`, который возвращает площадь прямоугольника.
- Создайте объект прямоугольника с произвольными значениями.
- Выведите его площадь.
- Измените ширину и высоту.
- Выведите новую площадь.

Пример вывода:

Площадь: 20

Новая площадь: 35

Домашнее задание

2. Класс Counter

Реализуйте класс `Counter`, который представляет собой простой счётчик.

- Счётчик должен начинаться с нуля.
- Предусмотрите методы для увеличения и уменьшения значения на единицу, при этом при каждой операции должно отображаться новое значение счётчика.
- Добавьте метод, возвращающий текущий результат.
- Проверьте работу счётчика, выполнив несколько операций.

Пример вывода:

```
Значение увеличено, текущее: 1
Значение увеличено, текущее: 2
Значение увеличено, текущее: 3
Значение уменьшено, текущее: 2
Текущее значение: 2
```




ВОПРОСЫ



Заключение

